



**KRUPAJAL ENGINEERING COLLEGE,
BHUBANESWAR**

DEPARTMENT OF CSE

**PROJECT REPORT ON
“LEARNING MANAGEMENT SYSTEM”**

GUIDED BY:

Prof. Aravinda Nanda(HOD)
Dept. of Computer Science & Eng.
Krupajal Engineering College
Bhubaneswar, Odisha

SUBMITTED BY

Binay Behera : 2221223007



CERTIFICATE

This is to certify that project report entitled as “**LEARNING MANAGEMENT SYSTEM**” submitted by our project team in partial fulfilment of the requirement for the final year project of BTech in department of Computer Science & Engineering of Krupajal Engineering College, is a record of candidates own work carried out by them under my supervision. The matter embodied in this report is original and not has been submitted for the final year project of any other collage.

SUPERVISOR:

Prof. Aravinda Nanda (HOD)

Dept. of Computer Science & Eng.

Krupajal Engineering College

Bhubaneswar, Odisha

EXTERNAL SIGNATURE:

DATE:



DECLARATION

We hereby declared that, this submission is our own work and that to that best of our knowledge and belief, it contains no material previously published or written by any other person nor material which to a substantial extent has been accepted for the final year project of any other institution of higher learning, except where due acknowledgement has been made in the test.

Binay Behera



ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who helped us to complete this project report on “**LEARNING MANAGEMENT SYSTEM**”.

We are very grateful and indebted to our project guided by **H.O.D Mr. Aravinda Nanda (CSE)** for providing their enduring patience, guidance and invaluable suggestion. They were the one who never let our moral down and always supported us through our thick and thin. They were the constant source of inspiration for us and took out most interest in our project.

We are also thankful to all the students for giving us their useful advice and immense co-operation. Their support made the working of this project very pleasant.

SUBMITTED BY:

Binay Behera

Project Report
On

**LEARNING
MANAGEMENT SYSTEM**

ABSTRACT

Web based applications play a main role in our day-today life and therefore, it is important to ensure the quality and reliability of web applications. With the sudden increase in use of web-based applications for online and distant learning, it is important to address existing issues in current Learning Management Systems (LMS), so that users can benefit from a better, uninterrupted learning experience. This work mainly contributes to understanding how the MERN stack can be efficiently used in building a reliable and secure LMS that will provide its services free of charge so that students are provided with a free and uninterrupted learning experience. An LMS that is equipped with user handling functionality, managing courses and course materials, and maintaining a library, has been the focus of the study that resulted in producing this paper. The work also includes reasoning as to why the MERN stack was selected for developing the proposed system.

Identification of the drawbacks of the existing system leads to the designing of computerized system that will be compatible to the existing system with the system Which is more user friendly and more GUI oriented. We can improve the efficiency of the system, thus overcome the drawbacks of the existing system.

Keywords-- ExpressJS, MERN, MongoDB, NodeJS, ReactJS

TABLE OF CONTENT

- Objective
- Existing System
- Proposed System

Chapter 1: FEASIBILITY STUDY

- Technical Feasibility
- Operational Feasibility
- Economical Feasibility

Chapter 2: REQUIREMENT SPECIFICATIONS

- Functional Requirement
- Number of Modules
- System Requirement
- Hardware Configuration
- Software Configuration
- Software Features
- SDLC Methodology

Chapter 3: DESIGN

- DFD Diagrams
- UML Diagrams
- ER Diagrams
- Database Design

Chapter 4 : TESTING

- Integration Testing
- Unit Testing

Chapter 5: CODING

Chapter 6: CONCLUSION

OBJECTIVE:

- ☐ The objective of the project is to develop a system that automates the processes and activities of a travel and tourism agency.
- ☐ The purpose is to design a system using which one can perform all operations related to traveling and sight-seeing.

EXISTING SYSTEM:

- ☐ In the present system a customer has to approach various agencies to find details of places and to book tickets.
- ☐ This often requires a lot of time and effort.
- ☐ A customer may not get the desired information from these offices and often the customer may be misguided.
- ☐ It is tedious for a customer to plan a particular journey and have it executed properly.

PROPOSED SYSTEM:

- ☐ The proposed system is a web based application and maintains a centralized repository of all related information.
- ☐ The system allows one to easily access the relevant information and make necessary travel arrangements.
- ☐ Users can decide about places they want to visit and make bookings online for travel and accommodation.

Chapter 1 :

FEASIBILITY STUDY:

FEASIBILITY REPORT:

Preliminary investigation examines project feasibility; the likelihood the system will be useful to the organization. The main objective of the feasibility study is to test the Technical, Operational and Economical feasibility for adding new modules and debugging old running system. All systems are feasible if they are given unlimited resources and infinite time. There are aspects in the feasibility study portion of the preliminary investigation:

- Technical Feasibility
- Operation Feasibility
- Economical Feasibility

TECHNICAL FEASIBILITY:

The technical issue usually raised during the feasibility stage of the investigation includes the following:

- Does the necessary technology exist to do what is suggested?
- Do the proposed equipments have the technical capacity to hold the data required to use the new system?
- Will the proposed system provide adequate response to inquiries, regardless of the number or location of users?
- Can the system be upgraded if developed?
- Are there technical guarantees of accuracy, reliability, ease of access and data security?

OPERATIONAL FEASIBILITY:

Proposed projects are beneficial only if they can be turned out into information systems, which will meet the organization's operating requirements. Operational feasibility aspects of the project are to be taken as an important part of the project implementation. Some of the important issues raised are to test the operational feasibility of a project includes the following: -

- Is there sufficient support for the management from the users?
- Will the system be used and work properly if it is being developed and implemented?
- Will there be any resistance from the user that will undermine the possible application benefits?

This system is targeted to be in accordance with the above-mentioned issues. Beforehand, the management issues and user requirements have been taken into consideration. So there is no question of resistance from the users that can undermine the possible application benefits.

The well-planned design would ensure the optimal utilization of the computer resources and would help in the improvement of performance status.

ECONOMIC FEASIBILITY:

A system can be developed technically and that will be used if installed must still be a good investment for the organization. In the economical feasibility, the development cost in creating the system is evaluated against the ultimate benefit derived from the new systems. Financial benefits must equal or exceed the costs. The system is economically feasible. It does not require any additional hardware or software.

Chapter 2 :

REQUIREMENT SPECIFICATIONS

The Learning Management System (LMS) is designed to facilitate seamless online education through efficient course management, interactive content delivery, and user engagement. The system aims to provide an intuitive platform for instructors to create courses and for learners to access educational content efficiently.

1. Functional Requirements:

The functional requirements define the core functionalities that the LMS must fulfill.

User Management:

- Users should be able to register and log in to the system.
- Authentication using secure credentials (e.g., email and password).
- Role-based access control: Admin, Instructor, and Student.
- Profile management, including updating personal information.
- Password reset and recovery via email.

Course Management:

- Instructors can create, update, and delete courses.
- Upload multimedia content such as videos, documents, and quizzes.
- Categorization of courses by topic, level, and tags.
- Students can browse, enroll, and access courses.
- Instructors can view enrolled students and track progress.

Lecture Management:

- Upload video lectures, PDFs, and related resources.
- Set lectures as free previews or restricted to enrolled students.
- Track progress and completion of lectures.

Progress Tracking and Reporting:

- Track course progress, including lectures completed and quizzes passed.
- Generate performance reports for instructors and students.
- Course completion certificates upon meeting criteria.

Payment and Subscription:

- Integration with secure payment gateways (like Stripe or PayPal).
- Support for free and paid courses.
- Generate invoices after successful payment.
- Apply discount codes during checkout.

2. Non-Functional Requirements:

The non-functional requirements define the system's performance and quality characteristics.

Performance:

- The system should handle high concurrency without affecting performance.
- Fast response time for loading courses and lectures.
- Efficient database queries to minimize latency.

Security:

- Secure user authentication using JWT.
- Data encryption during transmission (HTTPS).
- Role-based access control to protect data.
- Regular security audits and vulnerability assessments.

Usability:

- User-friendly interface with easy navigation.
- Mobile-responsive design for access from smartphones and tablets.
- Intuitive course creation and content management tools.

Scalability:

- Ability to handle a growing number of users and courses.
- Efficient media storage using cloud solutions (like Cloudinary).
- Modular architecture to add new features easily.

Reliability:

- Regular backups of user data and course content.
- High availability and minimal downtime.
- Robust error handling and user-friendly error messages.

Maintainability:

- Well-structured and modular code for easy updates.
- Documentation of code and features for developers and users.
- Automated testing for new features to ensure reliability.

3. System Requirements:

Hardware Requirements:

- **Server:** High-performance server with SSD storage.
- **Processor:** Quad-core or higher for optimal performance.
- **RAM:** Minimum 8 GB.
- **Storage:** At least 100 GB (expandable based on content volume).
- **Network:** High-speed internet connection for video streaming.

Software Requirements:

- **Operating System:** Linux (Ubuntu) or Windows Server.
- **Web Server:** NGINX or Apache.
- **Database:** MongoDB for efficient data management.
- **Backend Framework:** Node.js with Express.js for API management.
- **Frontend Framework:** React.js for a dynamic user interface.
- **Middleware:** Redux for state management.
- **Package Manager:** npm or yarn for dependency management.
- **Version Control:** Git for version tracking and collaboration.

4. User Interface Requirements:

- Simple and consistent UI/UX for a seamless experience.
- Accessible design with support for screen readers.
- Multi-language support (if applicable).
- Dark mode for a comfortable viewing experience.

5. Database Requirements:

- Store user details, course information, lectures, assessments, and media links.
- Efficient indexing for faster search and retrieval.
- Relations between courses, instructors, and students.
- Backup strategy to prevent data loss.

6. Integration Requirements:

- Payment Gateway Integration: Stripe/PayPal for payment processing.
- Cloud Storage: Cloudinary for media uploads and streaming.
- Email Service: Nodemailer for sending notifications.
- Third-party APIs: For enhanced functionalities like video processing.

7. Reporting and Analytics:

- Track course popularity and enrollment trends.
- Analyze user engagement, including video watch time and quiz performance.
- Generate custom reports for instructors and admins.

Software Features:

The **Learning Management System (LMS)** is designed to offer a comprehensive, user-friendly, and interactive learning environment. The software incorporates various features to facilitate learning, content management, and user engagement. Below are the key features categorized by user roles and system functionalities:

1. User Management:

- **User Authentication:**
 - Secure login and registration using JWT (JSON Web Tokens).
 - Role-based access control (Admin, Instructor, Student).
- **Profile Management:**
 - Users can update their personal information and change their profile picture.
 - Instructors and students can view their profile details, including enrolled or created courses.
- **Password Management:**
 - Ability to reset passwords via email verification.

2. Course Management:

- **Course Creation and Editing (For Instructors):**
 - Add, edit, or delete courses with a detailed description, syllabus, and tags.
 - Upload course images and introductory videos.
- **Course Enrollment (For Students):**
 - Enroll in available courses.
 - View enrolled courses in the user dashboard.
- **Course Progress Tracking:**
 - Track completed lectures, assignments, and quizzes.
 - Display course completion percentage.

3. Lecture and Content Management:

- **Lecture Upload:**
 - Upload video lectures, PDFs, and other materials.
 - Manage lecture titles, descriptions, and resources.
- **Content Accessibility:**
 - Instructors can mark lectures as **free preview** or **premium**.
 - Students can view lectures based on enrollment status.
- **Video Streaming and Management:**
 - Use of cloud-based storage (like Cloudinary) for video uploads.
 - Efficient video streaming for smooth playback.

4. Content Organization:

- **Course Categories and Tags:**
 - Categorize courses based on subjects or difficulty levels.
 - Add tags for better discoverability.
- **Curriculum Structuring:**
 - Arrange lectures and topics systematically.
 - Set prerequisites and sequential learning paths.

5. Payment and Subscription:

- **Payment Gateway Integration:**
 - Support for secure online payments using payment APIs (like Stripe or PayPal).
- **Subscription Models:**
 - Offer free, one-time payment, and subscription-based courses.
- **Invoice Generation:**
 - Automatically generate receipts after successful payment.
- **Discounts and Coupons:**
 - Apply promotional codes during checkout.

6. Notifications and Updates:

- **Email Notifications:**
 - Notify users about new courses, updates, or announcements.
- **In-app Alerts:**
 - Notify users about enrollment, lecture updates, and assignments.
- **Push Notifications:**
 - Send reminders about upcoming classes or deadlines.

7. Performance and Analytics

- **User Activity Tracking:**
 - Monitor student progress, quiz scores, and lecture completion rates.
- **Course Analytics:**
 - Track the number of enrollments, course ratings, and feedback.
- **Instructor Dashboard:**
 - Visual representation of course engagement and revenue generation.

8. Search and Filter Options:

- **Advanced Search:**
 - Search courses by title, instructor, category, or tags.
- **Filtering:**
 - Filter courses by difficulty level (Beginner, Intermediate, Advanced).
- **Sorting:**
 - Sort courses based on popularity, ratings, or newest first.

9. Data Backup and Security:

- **Automated Data Backup:**
 - Regularly back up course content, user data, and progress.
- **Secure Data Handling:**

- Use of HTTPS and encryption for data transmission.
- **Authorization and Authentication:**
 - Role-based access to prevent unauthorized actions.

10. Admin Panel:

- **User Management:**
 - Add, edit, or remove users.
 - View user statistics and course engagement.
- **Course Moderation:**
 - Approve or remove courses and content uploaded by instructors.
- **Report Management:**
 - Generate reports for user engagement, financial transactions, and content popularity.
- **Content Moderation:**
 - Review and moderate discussions and course comments.

11. Responsive and User-Friendly Interface:

- **Mobile Compatibility:**
 - Fully responsive design for accessibility on smartphones and tablets.
- **Intuitive Navigation:**
 - Clean and straightforward interface for all user types.
- **Accessibility Options:**
 - Support for screen readers and keyboard navigation.

SDLC Methodology:

The development of the Learning Management System (LMS) follows the Agile Software Development Life Cycle (SDLC) methodology. Agile is chosen for its flexibility, iterative development process, and ability to accommodate changes based on user feedback. The methodology is divided into the following phases:

1. Requirement Gathering:

During this phase, the development team collaborates with stakeholders to gather comprehensive requirements for the LMS. The primary objectives are to:

- Identify the core features and functionalities needed, such as user authentication, course management, content uploading, assessment, and progress tracking.
- Understand the user roles (Admin, Instructor, Student) and their specific requirements.
- Conduct meetings with educators, students, and administrators to get insights into their expectations.
- Document the functional and non-functional requirements.
- Prioritize the features based on their importance and user needs.

2. Planning:

This phase involves translating the gathered requirements into a strategic plan for development.

Key activities include:

- Designing the System Architecture:
 - Use of the MERN Stack (MongoDB, Express.js, React.js, Node.js) for a full-stack solution.
- Creating a Database Schema:
 - Design database models for courses, users, lectures, quizzes, and other entities.
- Preparing the Project Timeline:
 - Split the project into multiple sprints, each focusing on a specific set of features.
- Resource Allocation:
 - Assign roles and responsibilities to the team members, such as frontend, backend, and database management.

3. Development:

This phase involves building the LMS according to the plans formulated in the previous step. It follows an iterative approach with regular feedback loops.

- Backend Development:
 - Set up the server using Node.js and Express.js.
 - Implement API endpoints for course management, lecture uploads, and user interactions.
 - Integrate MongoDB for database management.
- Frontend Development:
 - Develop user interfaces using React.js and Tailwind CSS.
 - Implement client-side routing with React Router.
- API Integration:
 - Connect frontend components with backend APIs for data exchange.

- Testing during Development:
 - Write unit tests to verify the functionality of components and APIs.
 - Conduct manual testing of features within the development environment.
- Regular Code Reviews:
 - Ensure code quality and maintainability through peer reviews and code optimization.

4. Testing:

Testing is crucial to ensure that the system works as expected and meets user requirements. The testing phase involves:

- Unit Testing:
 - Test individual components and functions to ensure they work independently.
- Integration Testing:
 - Verify the interaction between different modules, such as user registration and course enrollment.
- System Testing:
 - Test the entire LMS as a whole to ensure seamless integration and performance.
- User Acceptance Testing (UAT):
 - Involve a group of real users to test the application in a simulated environment.
- Bug Fixing and Optimization:
 - Fix identified bugs and optimize the system for better performance.

5. Deployment:

After successful testing, the LMS is deployed to a production environment. The deployment process includes:

- Hosting the Backend:
 - Use cloud services like AWS, DigitalOcean, or Heroku for hosting the server.
- Deploying the Frontend:
 - Use platforms like Vercel or Netlify for hosting the React application.
- Database Configuration:
 - Host the MongoDB database on a cloud-based service like MongoDB Atlas.
- Domain Configuration and SSL:
 - Set up a custom domain and enable HTTPS for security.
- CI/CD Pipelines:
 - Automate the build, test, and deployment processes using tools like GitHub Actions or Jenkins.
- Version Control:
 - Maintain version history using Git and deploy the latest stable release.

6. Maintenance:

Maintenance is essential to ensure the LMS remains functional, secure, and up-to-date. Key maintenance tasks include:

- Bug Fixes:
 - Monitor error logs and user feedback to identify and resolve issues promptly.
- Feature Enhancements:
 - Gather feedback and incorporate new features based on user needs.

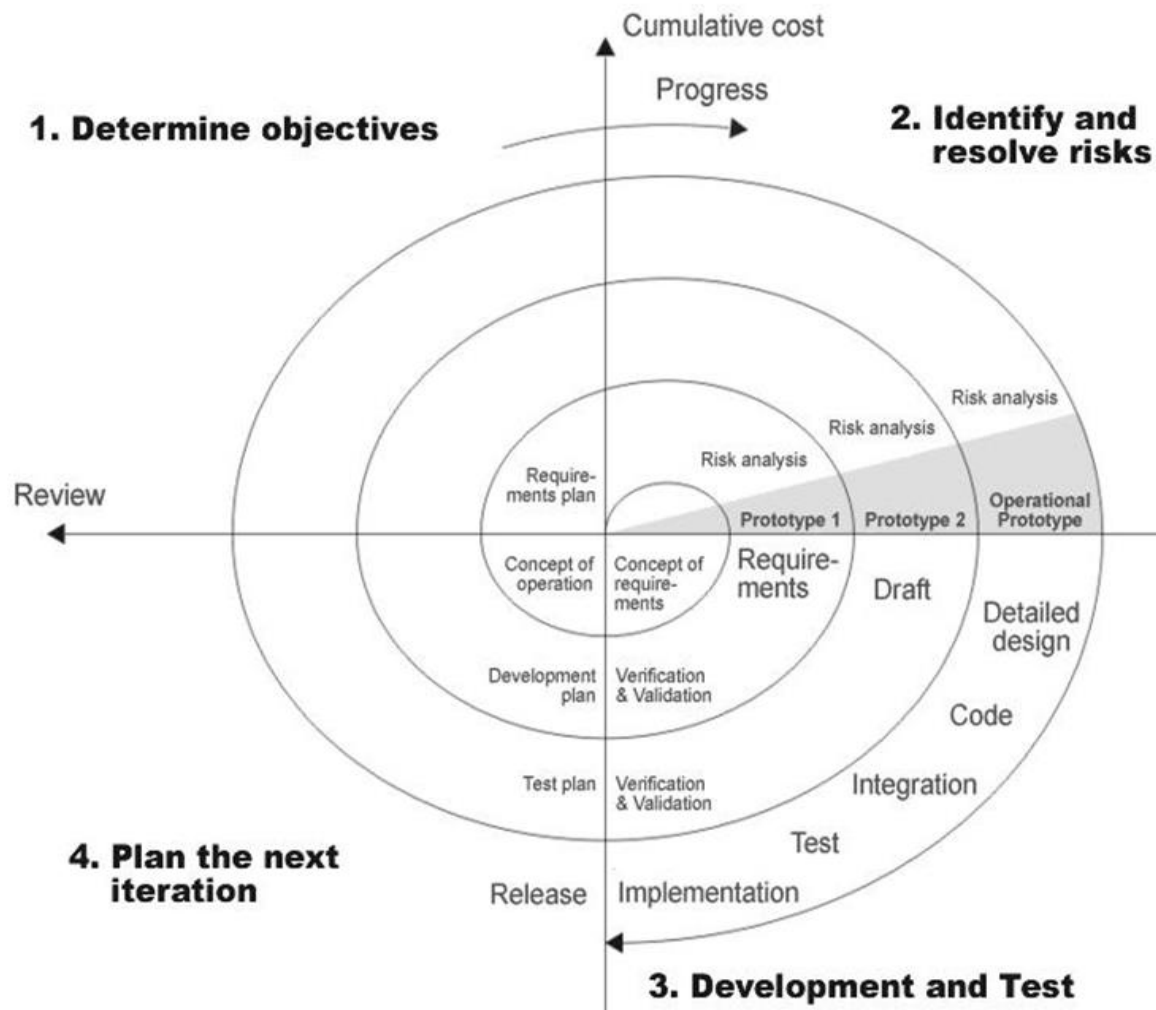
- Performance Optimization:
 - Regularly update the codebase and perform performance tuning.
- Security Updates:
 - Patch vulnerabilities and update libraries to prevent security breaches.
- Data Backup:
 - Schedule automated backups to ensure data safety and recovery.

7. Iteration:

Agile development encourages continuous improvement. In the iteration phase:

- Retrospective Analysis:
 - Analyze the project's progress and performance after each sprint.
- Incorporating Feedback:
 - Implement feedback received from users and stakeholders.
- Continuous Improvement:
 - Make iterative updates to add new features and enhance existing ones.

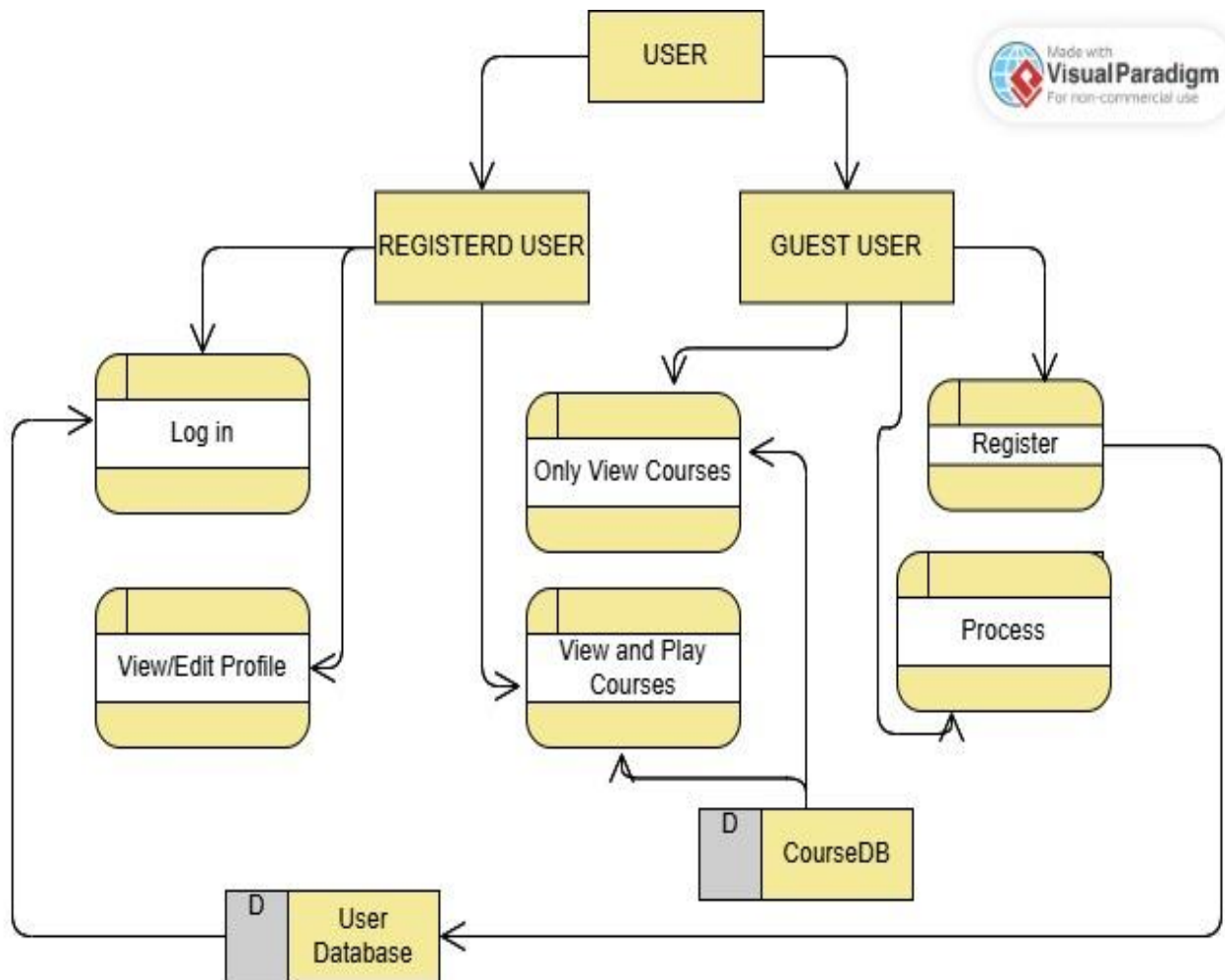
The following diagram shows how a spiral model acts like:



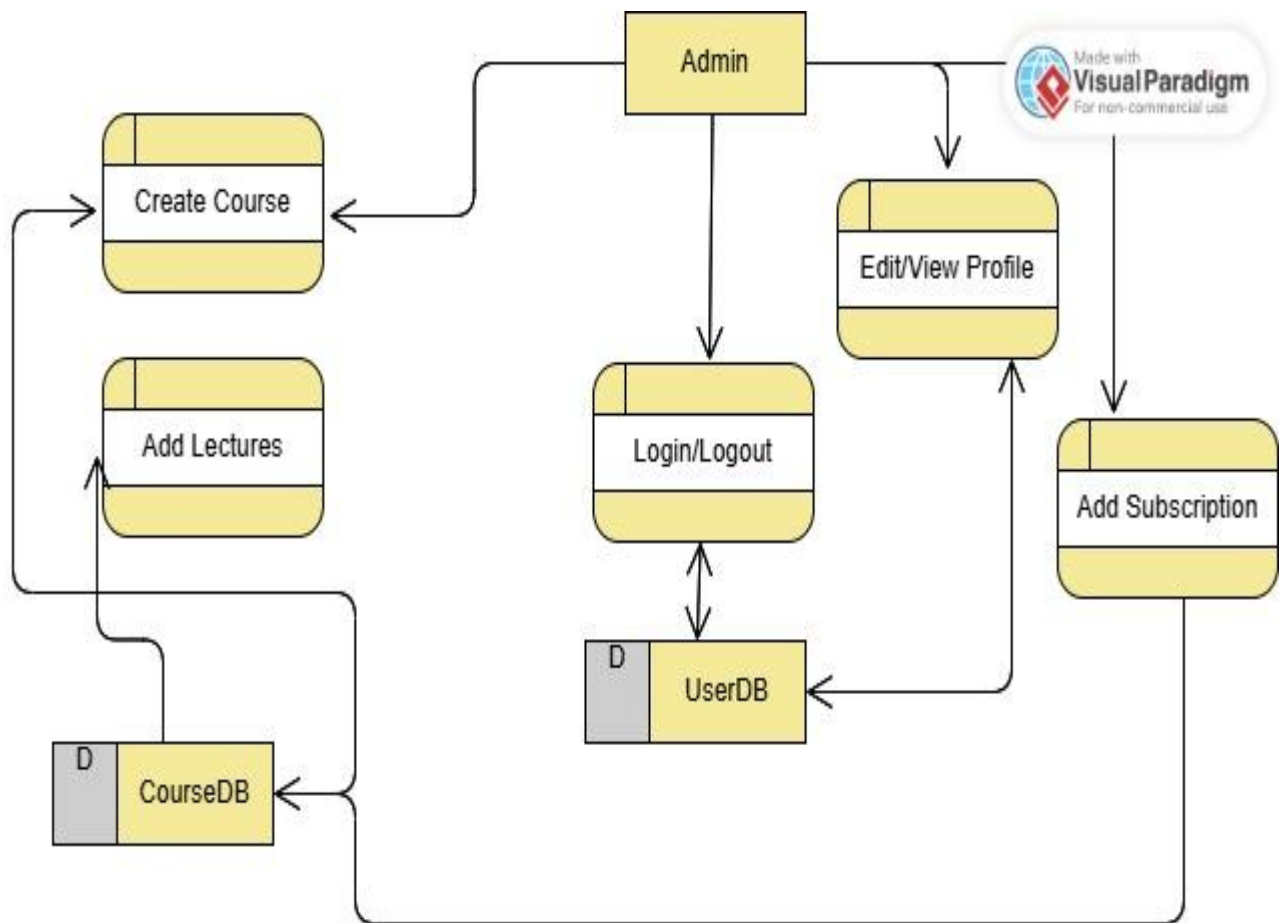
Chapter-3: Design

DFD (Context level Diagram):

1.Users

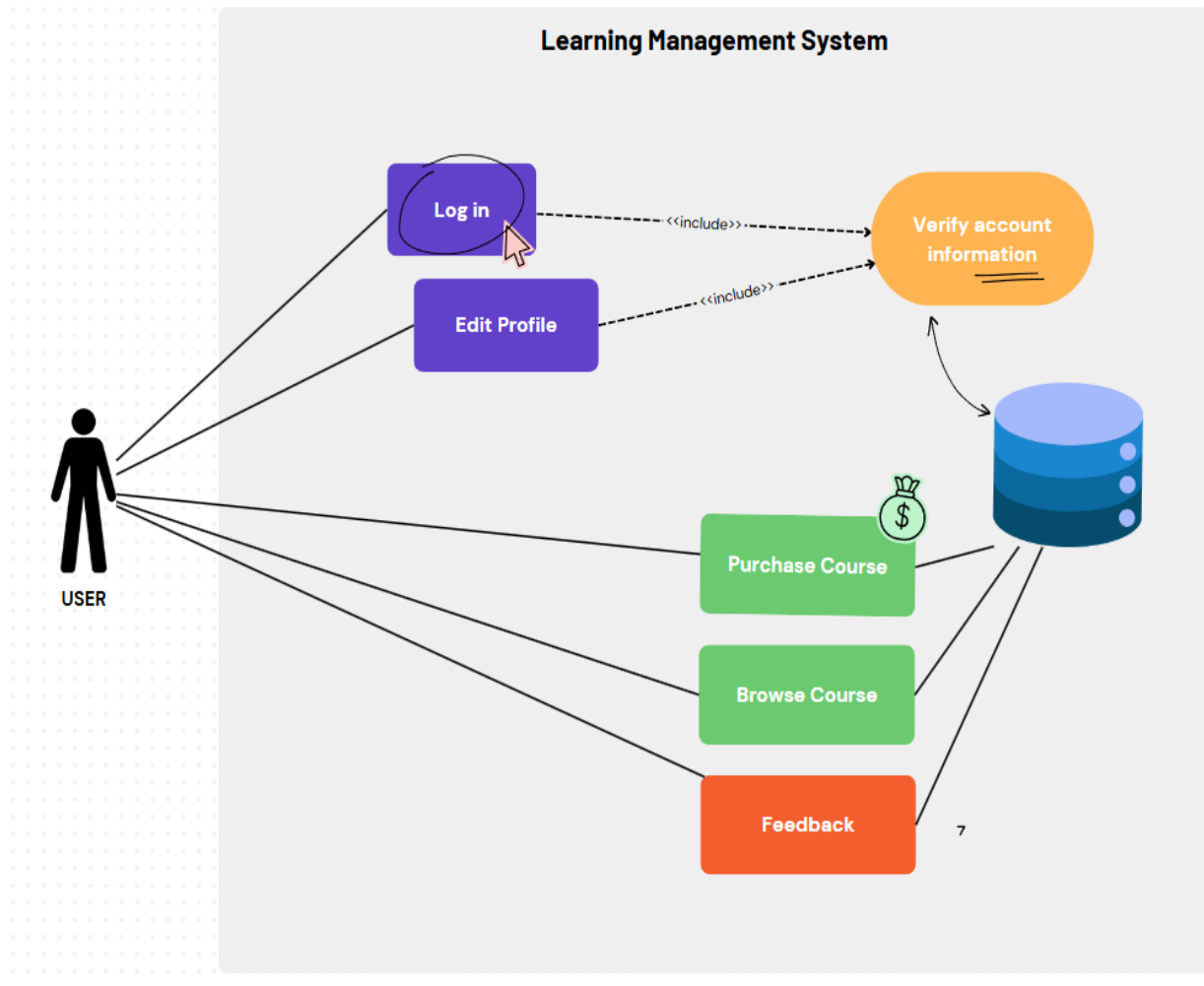


b. Admin

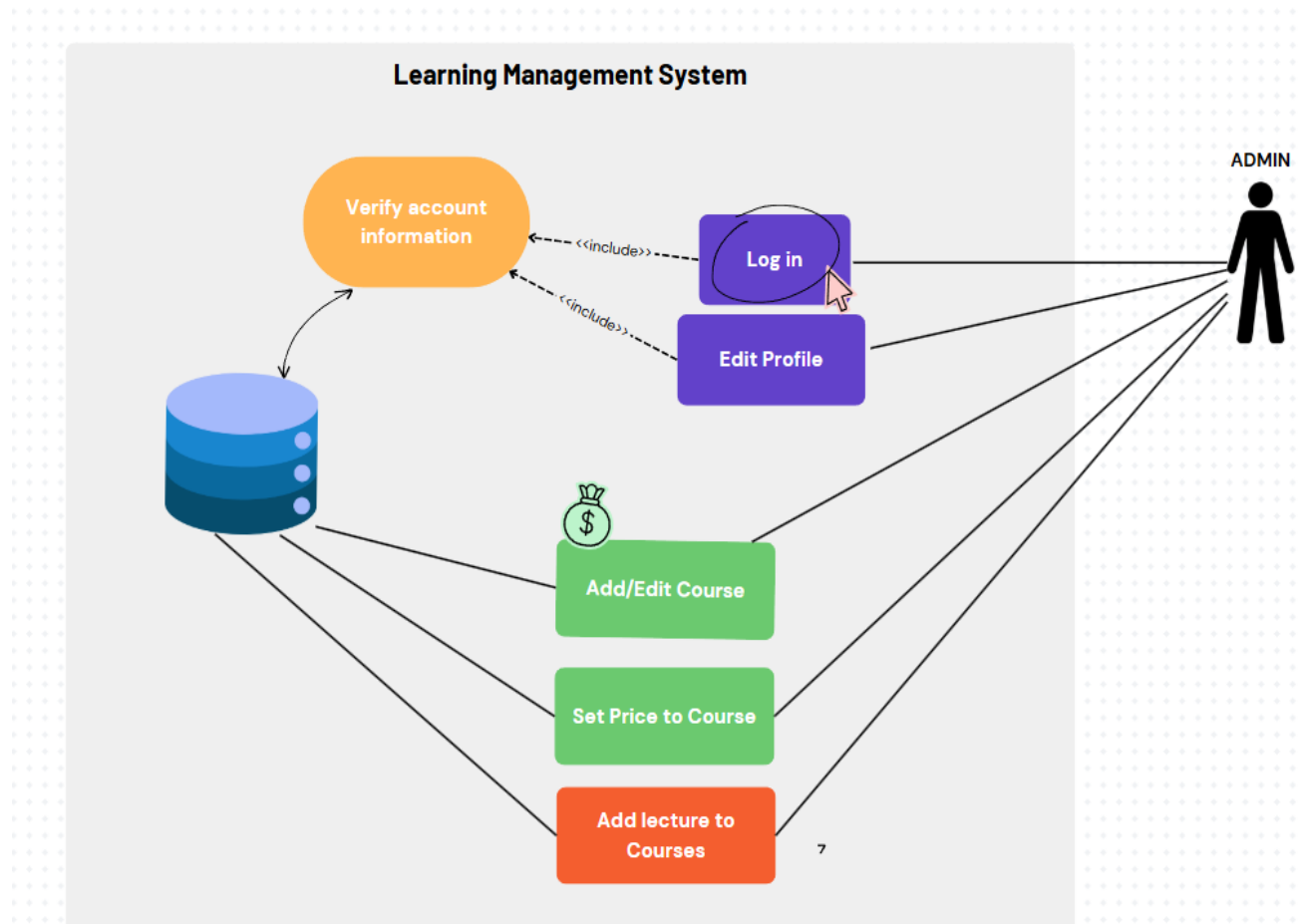


UML DIAGRAM

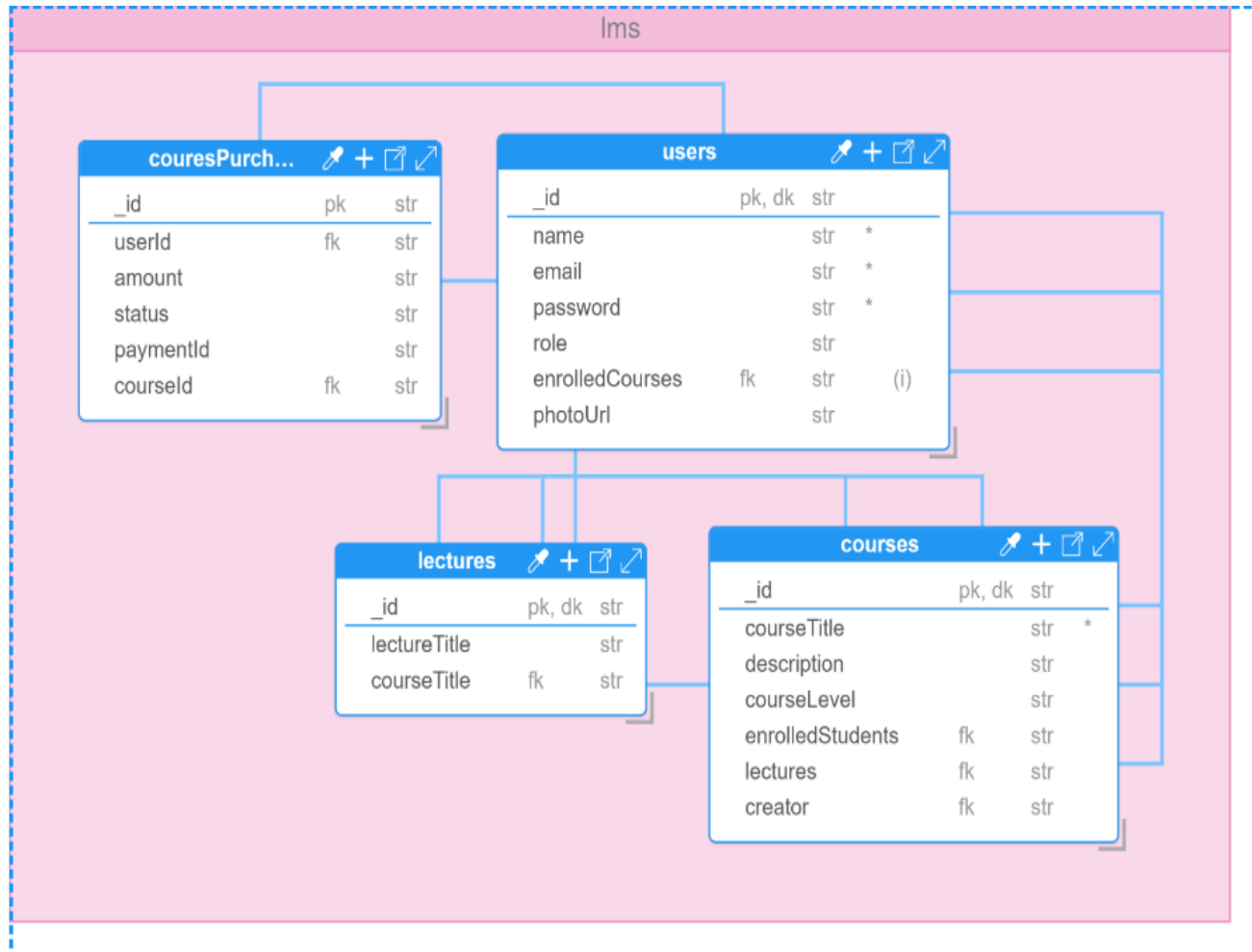
1.User



2.Admin



Database Design



The data in the system has to be stored and retrieved from database. Designing the database is part of system design. Data elements and data structures to be stored have been identified at analysis stage. They are structured and put together to design the data storage and retrieval system.

A database is a collection of interrelated data stored with minimum redundancy to serve many users quickly and efficiently. The general objective is to make database access easy, quick, inexpensive and flexible for the user. Relationships are established between the data items and unnecessary data items are removed. Normalization is done to get an internal consistency of data and to have minimum redundancy and maximum stability. This ensures minimizing data storage required, minimizing chances of data inconsistencies and optimizing for updates

Chapter 4:

TESTING

Testing is a critical phase in the **software development lifecycle (SDLC)** that ensures the **Learning Management System (LMS)** functions as expected, is reliable, and meets user requirements. Testing helps identify bugs, performance issues, and integration flaws, allowing developers to address them before deployment. In this chapter, we focus on two primary testing techniques: **Integration Testing and Unit Testing**.

1. Integration Testing:

Definition:

Integration testing is a phase where individual modules of the LMS are combined and tested as a group. The primary goal is to verify that integrated components work together as expected.

Objective:

- To identify and fix issues that may arise when multiple components interact.
- To ensure that data flows correctly between different modules, such as between the user authentication module and the course management module.

Approach:

- Top-Down Integration Testing: Start by testing higher-level modules and progressively move to lower-level modules.
- Bottom-Up Integration Testing: Start by testing lower-level modules and integrate them progressively with higher-level modules.
- Sandwich Integration Testing: Combines both top-down and bottom-up approaches.

Tools Used:

- Postman: For testing API endpoints.
- Jest with React Testing Library: To test UI components in React.
- Supertest: For testing HTTP requests and responses in Node.js.

Test Cases:

- User Registration and Authentication:
 - Verify that the registration API works seamlessly with the database.
 - Test successful and failed login attempts.
- Course Creation and Enrollment:
 - Test the integration between course creation forms and the database.
 - Check whether enrolled students are linked correctly to the respective courses.
- Lecture Upload and Viewing:
 - Ensure that uploaded lectures are linked to the correct course.
 - Verify video streaming functionality using cloud integration.
- Payment Gateway Integration:
 - Test the integration between the LMS and the payment provider (e.g., Stripe).
 - Simulate successful and failed transactions.

Outcome:

Integration testing ensures that the various modules of the LMS work cohesively without causing system crashes or data inconsistencies.

2.Unit Testing:

Definition:

Unit testing involves testing individual components or functions of the LMS in isolation. The purpose is to ensure that each unit of the application performs as expected.

Objective:

- To validate that individual functions, methods, and components work correctly.
- To identify bugs early in the development process.
- To ensure code quality and maintainability.

Approach:

- Test-Driven Development (TDD): Writing tests before writing the actual code to ensure coverage.
- Mocking: Using mock data to simulate different scenarios.
- Isolated Testing: Testing components independently without relying on external data or APIs.

Tools Used:

- Jest: For unit testing in JavaScript and React.
- Mocha and Chai: For backend testing.
- Supertest: For API testing in Node.js.
- Enzyme: For testing React components.

Test Cases:

- User Authentication:
 - Test login functionality with valid and invalid credentials.
 - Test the password reset feature.
- Course Module:
 - Test course creation with valid and invalid inputs.
 - Check the retrieval of course data from the database.
- Lecture Module:
 - Test lecture uploading with valid file formats.
 - Check lecture deletion and associated data removal.
- Progress Tracking:
 - Test the calculation of progress percentage based on completed lectures.
- Payment Module:
 - Simulate a payment transaction and verify the response.
 - Check for payment success and failure scenarios.

Chapter-5:

CODING PART

1.Server Creation Code

```
import express from 'express'
import dotenv from 'dotenv'
import cors from "cors"
import connectDB from './database/db.js';
import userRoute from './routes/user.route.js'
import courseRoute from './routes/course.route.js'
import mediaRoute from './routes/media.route.js'
import cookieParser from 'cookie-parser';
dotenv.config({});

const app = express();
const PORT = process.env.PORT || 3000;

app.use(express.json());
app.use(cookieParser());
app.use(cors({
  origin : "http://localhost:5173",
  credentials : true
}));

app.use('/api/v1/media' , mediaRoute)
app.use('/api/v1/user',userRoute);
app.use('/api/v1/course',courseRoute);

app.listen(PORT , ()=>{
  connectDB();
  console.log(`Server litsen at port ${PORT}`);

});
```

2.Database creation

```
import mongoose from "mongoose"
const connectDB = async ()=>{
  try {
    const connectionInstace = await mongoose.connect(process.env.MONGO_URI);
```

```

        console.log("Mongodb Connected !! DB HOST: ",connectionInstace.connection.host);

    } catch (error) {
        console.log("Error Occured" , error);
        process.exit(1);
    }
}

export default connectDB;

```

3.Backend Dependencies:

```

{
  "name": "server",
  "version": "1.0.0",
  "main": "index.js",
  "type": "module",
  "scripts": {
    "dev": "nodemon index.js"
  },
  "keywords": [],
  "author": "Binay",
  "license": "ISC",
  "description": "",
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cloudinary": "^2.5.1",
    "cookie-parser": "^1.4.7",
    "cors": "^2.8.5",
    "dotenv": "^16.4.7",
    "express": "^4.21.2",
    "jsonwebtoken": "^9.0.2",
    "mongoose": "^8.9.5",
    "multer": "^1.4.5-lts.1",
    "nodemon": "^3.1.9"
  }
}

```

4.Frontend dependencies :

```

{
  "name": "client",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {

```

```
"dev": "vite",
"build": "vite build",
"lint": "eslint .",
"preview": "vite preview"
},
"dependencies": {
"@radix-ui/react-avatar": "^1.1.2",
"@radix-ui/react-dialog": "^1.1.6",
"@radix-ui/react-dropdown-menu": "^2.1.5",
"@radix-ui/react-label": "^2.1.1",
"@radix-ui/react-progress": "^1.1.2",
"@radix-ui/react-select": "^2.1.6",
"@radix-ui/react-slot": "^1.1.1",
"@radix-ui/react-switch": "^1.1.3",
"@radix-ui/react-tabs": "^1.1.2",
"@reduxjs/toolkit": "^2.5.1",
"axios": "^1.8.4",
"class-variance-authority": "^0.7.1",
"clsx": "^2.1.1",
"lucide-react": "^0.471.0",
"next-themes": "^0.4.4",
"react": "^18.3.1",
"react-dom": "^18.3.1",
"react-quill": "^2.0.0",
"react-redux": "^9.2.0",
"react-router-dom": "^7.1.5",
"react-spinners": "^0.15.0",
"sonner": "^1.7.2",
"tailwind-merge": "^2.6.0",
"tailwindcss-animate": "^1.0.7"
},
"devDependencies": {
"@eslint/js": "^9.17.0",
"@types/react": "^18.3.18",
"@types/react-dom": "^18.3.5",
"@vitejs/plugin-react": "^4.3.4",
"autoprefixer": "^10.4.20",
"eslint": "^9.17.0",
"eslint-plugin-react": "^7.37.2",
"eslint-plugin-react-hooks": "^5.0.0",
"eslint-plugin-react-refresh": "^0.4.16",
"globals": "^15.14.0",
"postcss": "^8.4.49",
"tailwindcss": "^3.4.17",
"vite": "^6.0.5"
}
```

```
}
```

5.JWT Authentication(JSON WEB TOKEN):

```
import jwt from "jsonwebtoken"
```

```
const isAuthenticated = async (req,res,next) => {  
  try {  
    const token = req.cookies.token;  
    if(!token){  
      return res.status(401).json({  
        message:"User not authenticated",  
        success: false  
      })  
    }  
    const decode = await jwt.verify(token, process.env.SECRET_KEY);  
    if(!decode){  
      return req.statu(401).json({  
        message:"invalid token",  
        success: false  
      })  
    }  
  
    req.id = decode.userId;  
    next();  
  } catch (error) {  
    console.log(error);  
  
  }  
}  
export default isAuthenticated;
```

6.Course Model:

```
import mongoose from "mongoose"
```

```
const courseSchema = new mongoose.Schema({  
  courseTitle:{  
    type:String,  
    required:true  
  },  
  subTitle: {type:String},  
  description:{ type:String},  
})
```

```

    category:{
      type:String,
      required:true
    },
    courseLevel:{
      type:String,
      enum:["Beginner", "Intermediate", "Advance"]
    },
    coursePrice:{
      type:Number
    },
    courseThumbnail:{
      type:String
    },
    enrolledStudents:[
      {
        type:mongoose.Schema.Types.ObjectId,
        ref:'User'
      }
    ],
    lectures:[
      {
        type:mongoose.Schema.Types.ObjectId,
        ref:"Lecture"
      }
    ],
    creator:{
      type:mongoose.Schema.Types.ObjectId,
      ref:'User'
    },
    isPublished:{
      type:Boolean,
      default:false
    }
  }, {timestamps:true});

export const Course = mongoose.model("Course", courseSchema);

```

7.USER MODEL:

```

import mongoose from "mongoose";

const userSchema = new mongoose.Schema({
  name : {

```

```

        type : String,
        required : true
    },
    email:{
        type : String ,
        required : true
    },
    password : {
        type: String ,
        required : true
    },
    role : {
        type : String,
        enum:["instructor" , "student"],
        default: "student"
    },
    enrolledCourses:[
        {
            type: mongoose.Schema.Types.ObjectId,
            ref:'Course'
        }
    ],
    photoURL :{
        type: String,
        default:""
    },
    },{timestamps:true});

export const User = mongoose.model("User" , userSchema);

```

8. USER ROUTS:

```

import express from "express"
import { getUserProfile, login, logout, register, updateProfile } from
"./controllers/user.controller.js";
import isAuthenticated from "../middlewares/isAuthenticated.js";
import upload from "../utils/multer.js";
const router = express.Router();

router.route('/register').post(register);
router.route('/login').post(login);
router.route('/logout').get(logout);
router.route('/profile').get(isAuthenticated,getUserProfile);
router.route('/profile/update').put(isAuthenticated,upload.single("profilePhoto"),updateProfile);

```

```
export default router;
```

9. USER CONTROLLER:

```
import { User } from "../models/user.model.js";
import bcrypt from "bcryptjs";
import { generateToken } from "../utils/generateToken.js";
import { deleteMediaFromCloudinary, uploadMedia } from "../utils/cloudinary.js";
```

```
export const register = async (req, res) => {
  try {
    const { name, email, password } = req.body;

    if (!name || !email || !password) {
      return res.status(400).json({
        success: false,
        message: "All fields are required"
      });
    }

    const user = await User.findOne({ email });
    if (user) {
      return res.status(400).json({
        success: false,
        message: "User already exist with this email"
      });
    }

    const hashedPassword = await bcrypt.hash(password, 10);

    await User.create({
      name,
      email,
      password: hashedPassword
    });

    return res.status(201).json({
      success: true,
      message: "Account created successfully"
    });
  } catch (error) {
    console.log("Error in Register ", error);

    return res.status(500).json({
      success: false,
      message: "Failed to register"
    });
  }
};
```

```

    });
  }
};

export const login = async (req,res)=>{
  try {
    const {email , password} = req.body ;
    if(!email || !password){
      return res.status(400).json({
        success:false,
        message:"Something is missing"
      });
    };

    const user = await User.findOne({email});
    if(!user){
      return res.status(400).json({
        success:false,
        message:"Email doesn't exists"
      })
    };

    const checkPassword = await bcrypt.compare(password , user.password);
    if(!checkPassword){
      return res.status(400).json({
        success:false,
        message:"Incorrect Password"
      })
    };

    generateToken(res,user,`welcome back ${user.name}`);

  } catch (error) {
    console.log("Failed to Login" , error);
    return res.status(500).json({
      success : false ,
      message : "Failed to login"
    })
  }
}

export const logout = async (req,res)=>{
  try {
    return res.status(200).cookie("token","",{maxAge:0}).json({

```

```

        message:"Logout Sucessfully",
        success:true
    });
} catch (error) {
    console.log(error);
    return res.status(500).json({
        success: false,
        message:"failed to logout"
    })
}
};

export const getUserProfile = async (req,res) => {
    try {
        const userId = req.id;
        const user = await User.findById(userId).select("-password");
        if(!user){
            return res.status(404).json({
                message:"Profile not found",
                success:false
            })
        }
        return res.status(200).json({
            success:true,
            user
        });
    } catch (error) {
        console.log(error);
        return res.status(500).json({
            success: false,
            message:"failed to fetch user"
        })
    }
}

```

```

export const updateProfile = async (req,res) => {
    try {
        const userId = req.id;
        const {name} = req.body;
        const profilePhoto = req.file;

        const user = await User.findById(userId);
        if(!user){
            return res.status(404).json({
                message:"User not found",
                success:false
            })
        }
    }
}

```

```

    })
  }

  // extract public id of the old image from the url if it exists
  if(user.photoURL){
    const publicId = user.photoURL.split("/").pop().split(".")[0];
    deleteMediaFromCloudinary(publicId);
  }

  // upload new photo
  const cloudResponse = await uploadMedia(profilePhoto.path);
  const photoUrl = cloudResponse.secure_url;
  const updatedData = {name, photoURL : photoUrl};
  const updateUser = await User.findByIdAndUpdate(userId ,updatedData ,
{new:true}).select("-password");

  return res.status(200).json({
    success: true,
    user:updateUser,
    message:"Profile updated successfully"
  })
} catch (error) {
  console.log(error);

  return res.status(500).json({
    message:"Failed to update the profile",
    success: false
  })
}
}

```

Chapter 6:

CONCLUSION

In conclusion, the Learning Management System (LMS) built using the MERN stack (MongoDB, Express, React, Node.js) successfully streamlines the process of creating, managing, and delivering online courses. This platform offers a comprehensive solution for educators and learners alike by providing features such as course creation, lecture uploads, progress tracking, and user management.

By leveraging modern technologies, the LMS ensures a responsive and user-friendly interface, efficient data handling, and real-time updates. The integration of RESTful APIs and secure authentication guarantees a seamless and secure user experience.

Overall, this project demonstrates the potential of the MERN stack in building dynamic, scalable, and interactive web applications tailored to the evolving needs of online education. Future improvements may include incorporating advanced analytics, interactive quizzes, and enhanced collaboration tools to further enrich the learning experience.

The project also demonstrates effective use of API integration, authentication mechanisms, and responsive design, ensuring a seamless experience across different devices. By prioritizing user experience, data security, and performance optimization, this LMS serves as a reliable tool for both educators and learners.

In conclusion, the Learning Management System not only meets the current needs of online education but also lays a strong foundation for future innovations. As digital learning continues to evolve, this system can be expanded and improved to accommodate emerging educational technologies and practices, ensuring it remains relevant and impactful in the ever-changing educational landscape.

BIBLIOGRAPHY

MongoDB

MongoDB, Inc. (n.d.). Retrieved from <https://www.mongodb.com/docs/>

- This resource was used to understand the database structure, CRUD operations, and data modeling techniques essential for the backend of the LMS.

Express.js

OpenJS Foundation. (n.d.). Retrieved from <https://expressjs.com/>

- This documentation provided guidance on setting up the server, routing, and middleware integration for building a robust and scalable backend.

React

Meta Platforms, Inc. (n.d.). Retrieved from <https://react.dev/>

- React documentation was instrumental in developing interactive and dynamic user interfaces, including component management and state handling.

Node.js

OpenJS Foundation. (n.d.). Retrieved from <https://nodejs.org/en/docs/>

- This resource offered insights into building a backend server, handling HTTP requests, and integrating third-party packages.

Redux

Redux.js. (n.d.). Retrieved from <https://redux-toolkit.js.org/>

- This was used to manage global state effectively, allowing for smooth data flow between frontend components.

RTK-QUERY

Redux.js. (n.d.). Retrieved from <https://redux-toolkit.js.org/rtk-query/overview>

- RTK Query helped in handling API calls and caching responses, enhancing data synchronization between the client and server.

JWT-Authentication

Auth0. (n.d.). Retrieved from <https://auth0.com/docs>

- This resource provided valuable information on implementing secure user authentication and managing JSON Web Tokens (JWT) for session management.

Axios

Axios. (n.d.). Retrieved from <https://axios-http.com/>

- Used to handle HTTP requests efficiently within the application, allowing file uploads and form data submission.

Tailwind-CSS

Tailwind Labs. (n.d.). Retrieved from <https://tailwindcss.com/docs>

- Helped in designing a responsive and modern UI, making the application visually appealing and user-friendly.

Sonner:Toast

Sonner. (n.d.). Retrieved from <https://sonner.dev/>

- Integrated for displaying user-friendly notifications, enhancing the feedback mechanism of the application.